

- **Lab Title:** Lab 1B: PC System Fundamentals, Disk Sectors and Partition Analysis
- **Assignment Title:** Basic Computer Skills for Digital Forensics
- **Student Name:** Bashir Adam
- **Student ID:** 2025/FWSD/11509
- **Course Name:** SBT-DF201: Basic Computer Skills for Digital Forensics
- **Instructor Name:** Aminu Idris
- **Date of Submission:** 8 Aug, 2026
- **Version:** Version 2.1.1

platform used: Kali Linux

Lab Overview

This lab introduces the basic computer system knowledge required for beginner digital forensic work. Students will identify system hardware and software information, map where digital evidence can exist, calculate disk and partition sizes from sector information, inspect partition layout using safe read-only commands, explain the boot process, and identify file systems used by common operating systems.

Introduction

This lab explores the fundamentals of PC system architecture, disk storage, and partition analysis from a digital forensics perspective. The purpose was to gather and interpret real system evidence including hardware specifications, running processes, disk sector data, partition tables, boot configuration, and file systems using command-line tools rather than relying on assumptions or generic examples. Working entirely within a Linux/Kali environment, I identified my own system's processor, memory, and boot mode; investigated live process activity; manually calculated partition sizes from raw sector data; safely inspected my partition table without risking data modification; explained how MBR and GPT differ and how the boot process unfolds; and identified the file systems on my own disk. The overall aim was to connect number-level, byte-level evidence directly to forensic reasoning understanding not just what a tool reports, but why that information matters when reconstructing or verifying digital evidence.

Tools Used

Operating System: Kali GNU/Linux Rolling (kernel 7.0.12+kali-amd64), running on a Lenovo ThinkPad L440.

Commands used:

- `hostnamectl`, `lscpu`, `free -h` system specification and hardware identification
- `[ -d /sys/firmware/efi ]` — UEFI/Legacy BIOS detection
- `lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT`, `sudo fdisk -l`, `df -h` — storage and partition table information
- `ps -eo pid,comm,%cpu,%mem --sort=-%cpu` — running process investigation
- `sudo parted -l`, `blkid` — safe, read-only partition table cross-verification
- `lsblk -f`, `df -Th` — file system identification
- `python3` (manual calculation checks) — verifying sector-to-byte-to-GB conversions

Software used: Terminal (Kali default), text editor for report drafting (LibreOffice Writer). No disk-modifying or third-party forensic imaging tools were used, in line with the lab's read-only evidence-handling requirement.

A screenshot of a Kali Linux terminal window. The terminal shows the output of the 'hostnamectl' command, displaying system information such as Static hostname: localhost, Icon name: computer-laptop, Chassis: laptop, Machine ID, Boot ID, Operating System: Kali GNU/Linux Rolling, Kernel: Linux 7.0.12-kali-amd64, Architecture: x86-64, Hardware Vendor: Lenovo, Hardware Model: ThinkPad L440, Hardware SKU: LENOVO_MT_20AS_BU_Think_FM_ThinkPad L440, Firmware Version: J4ET93WW(1.93), Firmware Date: Wed 2018-05-30, and Firmware Age: By 2month 1w 4d. The terminal window has a dark background with light-colored text. The top of the window shows the title bar with standard Linux window controls and system status icons.

This gave me detailed processor information. My CPU is an **Intel(R) Core(TM) i5-4210M @ 2.60GHz**, architecture **x86_64**, with 4 CPUs total (2 cores per socket, 2 threads per core, 1 socket),

```
azad@localhost: ~  
$ lscpu  
Architecture:          x86_64  
CPU op-mode(s):        32-bit, 64-bit  
Address sizes:          39 bits physical, 48 bits virtual  
Byte Order:             Little Endian  
CPU(s):                 4  
On-line CPU(s) list:    0-3  
Vendor ID:              GenuineIntel  
Model name:             Intel(R) Core(TM) i5-4210M CPU @ 2.60GHz  
CPU family:             6  
Model:                  60  
Thread(s) per core:     2  
Core(s) per socket:     2  
Socket(s):              1  
Stepping:               3  
Microcode version:      0x28  
CPU(s) scaling MHz:     91%  
CPU max MHz:            3200.0000  
CPU min MHz:            800.0000  
BogoMIPS:               5188.18  
Flags:                  fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov  
pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe sy  
scall nx pdpeibg rdtscp lm constant_tsc arch_perfmon pebs bts  
rep_good nopl xtopology nonstop_tsc cpuid aperfperf pni pcl  
mulldq dtes64 monitor ds_cpl vmx est tm2 ssse3 sdbg fma cx16  
xtpr pdcm pcid sse4_1 sse4_2 movbe popcnt tsc_deadline_timer  
aes xsave avx f16c rdrand lahf_lm abm cpuid_fault epb pti ssb  
d_ibrs ibpb stibp tpr_shadow flexpriority ept vpid ept_ad fsg  
sbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid xsaveopt dt  
herm ida arat pln pts vmmi md_clear flush_l1d  
Virtualization features:  
Virtualization:         VT-x  
Caches (sum of all):  
L1d:                    64 KiB (2 instances)  
L1i:                    64 KiB (2 instances)  
L2:                     512 KiB (2 instances)  
L3:                     3 MiB (1 instance)
```

This showed my installed physical memory: 7.7 GiB total RAM, with 4.9 GiB used and 1.4 GiB free at the time of capture, plus a 7.9 GiB swap partition. My second result was **UEFI**, confirming my system boots using modern UEFI

```
azad@localhost: ~  
$ free -h  
              total        used        free      shared  buff/cache   available  
Mem:          7.7Gi        4.9Gi        1.4Gi        515Mi        2.2Gi        2.8Gi  
Swap:         7.9Gi        3.0Gi        5.0Gi  
  
azad@localhost: ~  
$ [ -d /sys/firmware/efi ] && echo "UEFI" || echo "Legacy BIOS"  
UEFI  
  
azad@localhost: ~  
$
```

A2. Capture storage information.

I ran three commands to capture storage information on my Kali system. `lsblk` showed my physical disk `/dev/sda` (465.8 GiB) is divided into three partitions: `sda1` (976M, vfat, mounted at `/boot/efi`), `sda2` (456.9G, ext4, mounted at `/`), and `sda3` (7.9G, swap). There's also `sr0`, an optical drive with no media.

`sudo fdisk -l` gave the detailed partition table. My disk is a SanDisk SDSSDH35 SSD, reporting 465.76 GiB / 500,107,862,016 bytes / 976,773,168 sectors, with a sector size of 512 bytes (logical and physical) and a GPT disklabel — consistent with my system booting UEFI, as confirmed in Part A1. The disk identifier (GUID) is B70B38FF-7899-4F64-9C96-17129FD24D92.

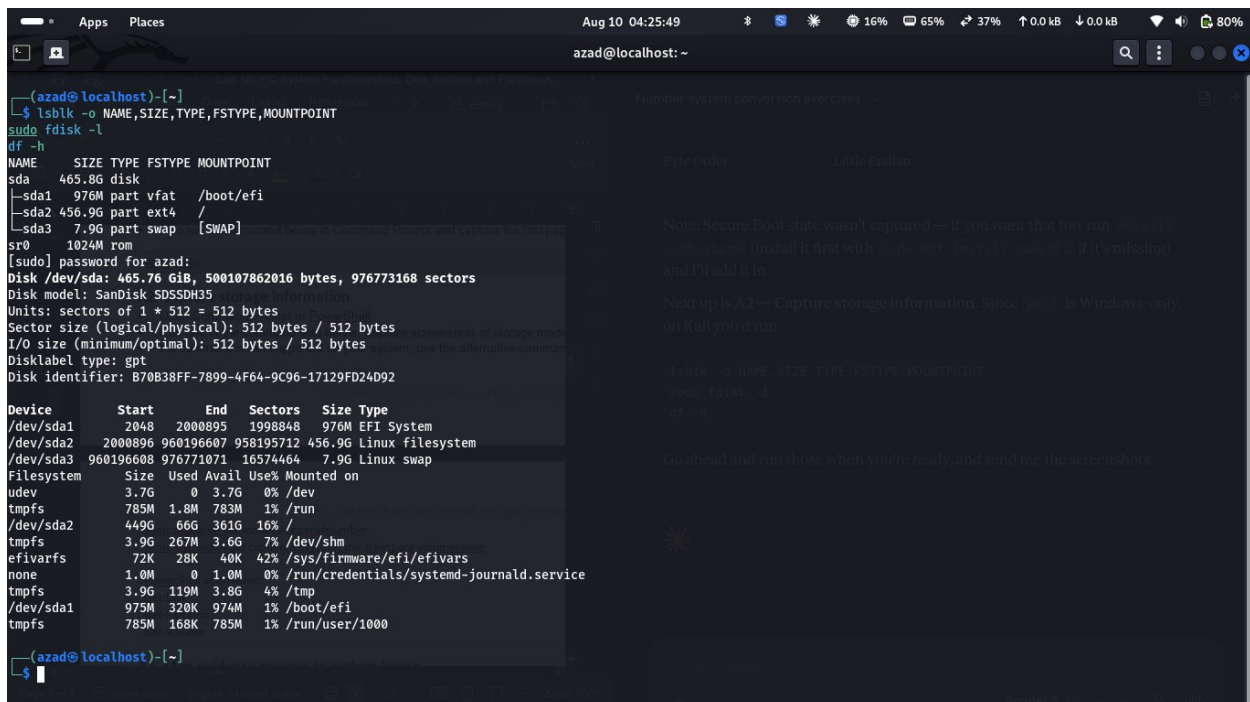
The partition table breaks down as follows:

`/dev/sda1`: start sector 2048, end 2000895, 1,998,848 sectors, ~976M, type EFI System

`/dev/sda2`: start sector 2000896, end 960196607, 958,195,712 sectors, ~456.9G, type Linux filesystem

`/dev/sda3`: start sector 960196608, end 976771071, 16,574,464 sectors, ~7.9G, type Linux swap

Finally, `df -h` showed the OS-reported mounted filesystem sizes: `/dev/sda2` reports 449G total, 66G used, 361G available (16% used) mounted at `/`, and `/dev/sda1` reports 975M total, 320K used, 974M available (1% used) mounted at `/boot/efi`.



```
(azad@localhost)-[~]
$ lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT
lsblk
$ sudo fdisk -l
fdisk
$ df -h
df -h
```

NAME	SIZE	TYPE	FSTYPE	MOUNTPOINT
sda	465.8G	disk		
_sda1	976M	part	vfat	/boot/efi
_sda2	456.9G	part	ext4	/
_sda3	7.9G	part	swap	[SWAP]
sr0	1024M	rom		

```
[sudo] password for azad:
Disk /dev/sda: 465.76 GiB, 500107862016 bytes, 976773168 sectors
Disk model: SanDisk SDSSDH35
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: B70B38FF-7899-4F64-9C96-17129FD24D92
```

Device	Start	End	Sectors	Size	Type
/dev/sda1	2048	2000895	1998848	976M	EFI System
/dev/sda2	2000896	960196607	958195712	456.9G	Linux filesystem
/dev/sda3	960196608	976771071	16574464	7.9G	Linux swap

Filesystem	Size	Used	Avail	Use%	Mounted on
udev	3.7G	0	3.7G	0%	/dev
tmpfs	785M	1.8M	783M	1%	/run
/dev/sda2	449G	66G	361G	16%	/
tmpfs	3.9G	267M	3.6G	7%	/dev/shm
efivarfs	72K	28K	40K	42%	/sys/firmware/efi/efivars
none	1.0M	0	1.0M	0%	/run/credentials/systemd-journald.service
tmpfs	3.9G	119M	3.8G	4%	/tmp
/dev/sda1	975M	320K	974M	1%	/boot/efi
tmpfs	785M	168K	785M	1%	/run/user/1000

```
(azad@localhost)-[~]
$
```

A3. Map evidence sources to system layers.

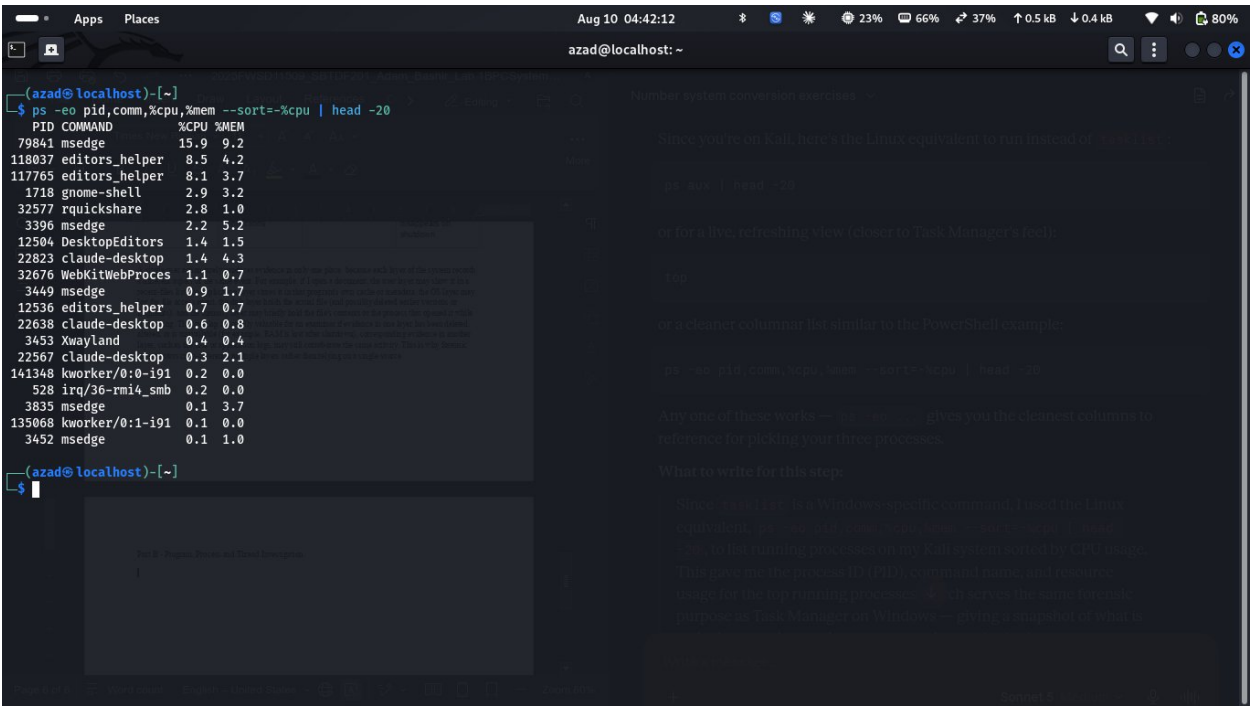
Layer / Source	Example Evidence	Where It May Be Found	Why It Matters
User layer	Opened files, typed commands, searched terms	Links a specific user account to actions taken on the system	Links a specific user account to actions taken on the system
Application layer	Browser cache/history, document metadata, chat logs	~/.mozilla/firefox/, ~/.config/, application-specific data folders	Shows what software created, opened, or modified the evidence
OS layer	System logs, installed packages, user account info	/var/log/, /etc/passwd, dpkg -l / apt logs	Shows system-level events, logins, and account activity
Disk layer	Deleted files, file fragments, partition structure	Unallocated space, /dev/sda1, /dev/sda2, /dev/sda3 (from my A2 output)	Supports file recovery and timeline reconstruction
Memory/process layer	Running processes, PIDs, open network connections	Live RAM, output of ps aux, netstat/ss	Shows live, volatile system behavior that disappears on shutdown

A single user action rarely produces evidence in only one place, because each layer of the system records a different aspect of the same event. For example, if I open a document, the user layer may show it in a recent-files list, the application layer stores it in that program's own cache or metadata, the OS layer may log the file access event, the disk layer holds the actual file (and possibly deleted earlier versions or fragments), and the memory layer may briefly hold the file's contents or the process that opened it while it's running. This overlap is actually valuable for an examiner if evidence in one layer has been deleted, altered, or is unavailable (for example, RAM is lost after shutdown), corresponding evidence in another layer, such as the disk or application logs, may still corroborate the same activity. This is why forensic investigators cross-reference multiple layers rather than relying on a single source.

Part B - Program, Process and Thread Investigation

B1. Observe running processes

I listed the top 20 running processes on my Kali system sorted by CPU usage, since tasklist is Windows-specific insted i used “ps -eo pid,comm,%cpu,%mem --sort=-%cpu | head -20”. The output showed a mix of browser processes, desktop environment components, and background system workers, giving a live snapshot of what was actively running on my machine at the time of capture.



The three processes

Process Name	PID	%CPU	%MEM	Likely Purpose
msedge	79841	15.9%	9.2%	Microsoft Edge browser process the highest CPU consumer at capture time, likely rendering active tabs/content.
gnome-shell	1718	2.9%	3.2%	The GNOME desktop environment's core shell process responsible for the graphical UI, window management, and desktop interactions.
claude-desktop	22823	1.4%	4.3%	A desktop application process (AI assistant client) shows as a user-launched application actively consuming CPU/memory resources.

B2. Explain the difference

- A program is a static file stored on disk for example, the executable for Microsoft Edge sitting in its installation directory. It doesn't do anything on its own until it's executed. A process is what's created when that program is actually run: the operating system allocates it memory, a PID, and system resources, and it becomes an active, running instance like msedge with PID 79841 in my output. A thread is a smaller unit of execution within a process; a single process can have multiple threads running concurrently to handle different tasks in parallel, sharing the same memory space. For example, a browser process like msedge likely uses separate threads internally to handle rendering, networking, and user input at the same time, even though they're all part of one PID.
- Process information is especially useful during malware analysis or live forensic triage because a process listing captures what's actually executing on a system at a specific moment something that can't be reconstructed later from disk alone. Malware often runs as a disguised or unusually named process, spawns unexpected child processes, or consumes abnormal CPU/memory. In my output, seeing multiple msedge and claude-desktop instances with varying resource use is normal browser/app behavior but an examiner trained to spot anomalies would look for a process with a suspicious name, no clear parent, unusually high resource use, or a PID tied to an unexpected network connection. Capturing this snapshot before shutdown matters because once the system powers off, this live process/memory evidence is gone only disk-based artifacts remain.

Part C - Disk Sector and Disk Size Calculation

C1. Reproduce the slide calculation

Since my disk is a modern SSD using GPT (not the old CHS geometry the slide example uses), fdisk reports the total sector count directly rather than sectors-per-track/heads/cylinders. From my Part A2 screenshot: /dev/sda has 976,773,168 sectors, at 512 bytes per sector.

Rather than use the slide's generic example values, I applied the same formula to my own physical disk, using the sector count and sector size reported by `sudo fdisk -l`

Item	Formula / Value	Answer
Total sectors		976,773,168
Total bytes	$976,773,168 \times 512$	500,107,862,016 bytes
Kilobytes	$500,107,862,016 / 1024$	488,386,584 KB
Megabytes	$488,386,584 / 1024$	476,940.02 MB
Gigabytes	$476,940.02 / 1024$	465.76 GB

C2. Calculate your own disk or partition size

Partition 1: /dev/sda1 (EFI System)

Sector count: 1,998,848

Sector size: 512 bytes

Size in bytes = $1,998,848 \times 512$ = 1,023,410,176 bytes

Size in KB = $1,023,410,176 / 1024$ = 999,424 KB

Size in MB = $999,424 / 1024$ = 976 MB

Size in GB = $976 / 1024$ = 0.95 GB

Partition 2: /dev/sda2 (Linux filesystem, root)

Sector count: 958,195,712

Sector size: 512 bytes

Size in bytes = $958,195,712 \times 512$ = 490,596,204,544 bytes

Size in KB = $490,596,204,544 / 1024$ = 479,097,856 KB

Size in MB = $479,097,856 / 1024$ = 467,869 MB

Size in GB = $467,869 / 1024$ = 456.9 GB

Partition 3: /dev/sda3 (Linux swap)

Sector count: 16,574,464

Sector size: 512 bytes

Size in bytes = $16,574,464 \times 512$ = 8,486,125,568 bytes

Size in KB = $8,486,125,568 / 1024$ = 8,287,232 KB

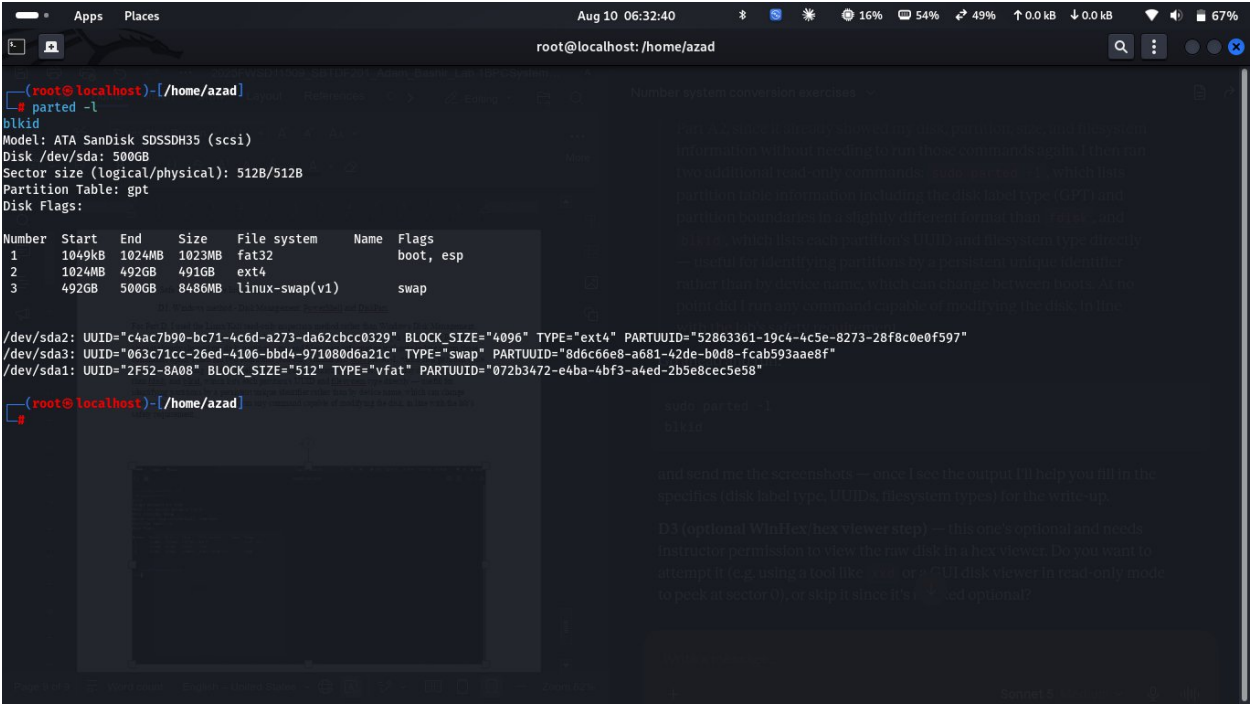
Size in MB = $8,287,232 / 1024$ = 8,093 MB

Size in GB = $8,093 / 1024$ = 7.9 GB

Part D - Safe Partition Table Inspection

D1. Linux/Kali method - read-only inspection

For Part D, I used the Linux/Kali read-only inspection method rather than Windows Disk Management, since my working environment is Kali. I reused my lsblk, sudo fdisk -l, and df -h output from Part A2, since it already showed my disk, partition, size, and filesystem information without needing to run those commands again. I then ran two additional read-only commands: sudo parted -l, which lists partition table information including the disk label type (GPT) and partition boundaries in a slightly different format than fdisk, and blkid, which lists each partition's UUID and filesystem type directly — useful for identifying partitions by a persistent unique identifier rather than by device name, which can change between boots. At no point did I run any command capable of modifying the disk, in line with the lab's safety requirement.



Partition	Filesystem	UUID	PARTUUID	Notes
/dev/sda1	vfat	2F52-8A08	072b3472-e4ba-4bf3-a4ed-2b5e8cec5e58	EFI System Partition, boot/esp flags
/dev/sda2	ext4	c4ac7b90-bc71-4c6d-a273-da62cbcc0329	52863361-19c4-4c5e-8273-28f8c0e0f597	Root filesystem
/dev/sda3	swap	063c71cc-26ed-4106-bbd4-971080d6a21c	8d6c66e8-a681-42de-b0d8-fcab593aae8f	Swap partition

Part E - MBR, GPT and Boot Process Explanation

E1. Explain the boot process

Stage	What Happens	Forensic Relevance
Power-on and POST	Hardware is initialized and checked	Hardware failure or tampering may affect evidence access
BIOS/UEFI firmware	Boot mode and firmware settings are applied	Boot mode explains partition style and boot artifacts
MBR/GPT or EFI boot	System locates bootable partition or EFI entry	Boot records may contain evidence of boot configuration
Boot loader/manager	OS selection and pre-kernel loading occurs	May show multi-boot or hidden OS behavior
Kernel and OS control	Operating system fully starts	Logs and user activity begin to accumulate

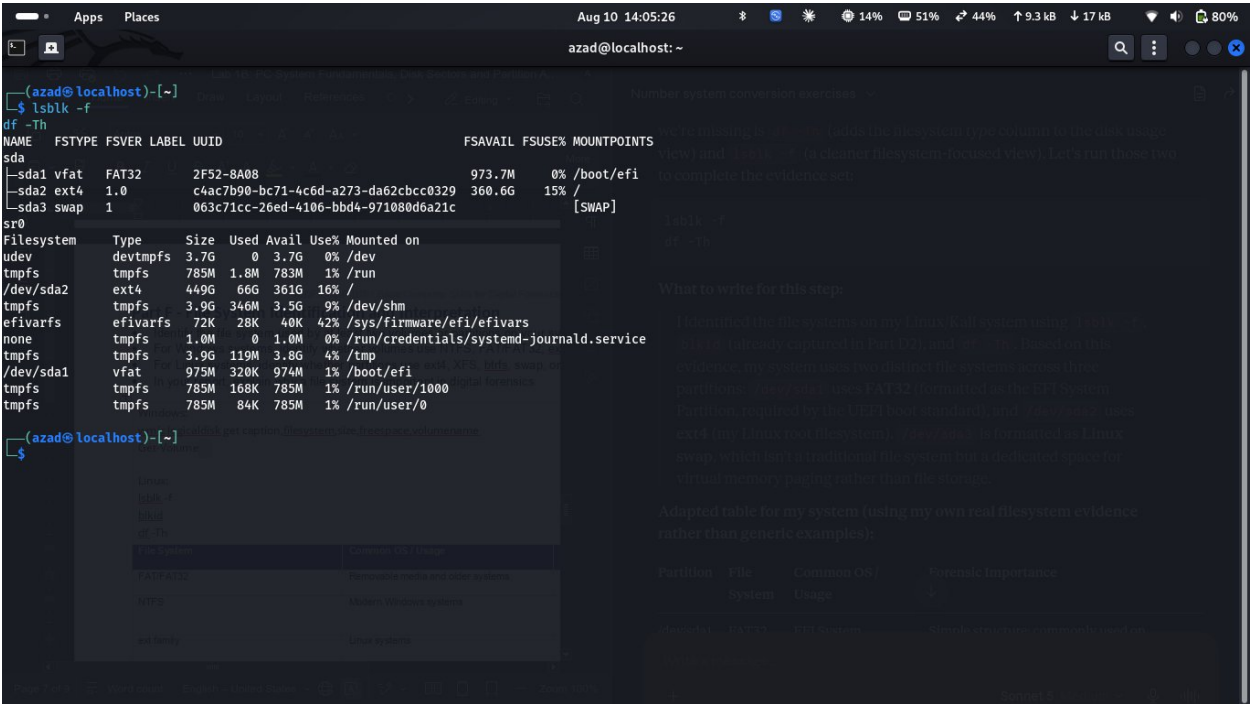
E2. Compare MBR and GPT

My own disk uses GPT, confirmed independently through `sudo fdisk -l` (Disklabel type: gpt) and `sudo parted -l` (Partition Table: gpt). This is consistent with the system booting UEFI rather than Legacy BIOS. From a forensic resilience standpoint, this is actually a small advantage: because GPT stores a backup partition table at the end of the disk, if the primary GPT header at the start of my disk were ever corrupted or maliciously altered, there's a redundant copy that could support recovery a protection that plain MBR does not offer, since MBR concentrates all partition information in a single sector with no built-in backup.

Feature	MBR	GPT
Age / design	Older partitioning scheme from the 1980s, paired with Legacy BIOS	Modern partitioning scheme used with UEFI systems
Size limitation	Commonly limited to ~2 TB	Supports very large disks (~9.4 ZB)
Partition information	Stored in the first sector along with boot code	Stored in primary and backup GPT headers plus partition entries
Forensic issue	Critical info concentrated in sector 0 damage there can corrupt the whole partition table	Backup GPT structures at the end of the disk allow recovery/validation if the primary header is damaged

Part F - File System Identification and Interpretation

I identified the file systems on my Linux/Kali system using lsblk -f, blkid (already captured in Part D2), and df -Th. Based on this evidence, my system uses two distinct file systems across three partitions: /dev/sda1 uses FAT32 (formatted as the EFI System Partition, required by the UEFI boot standard), and /dev/sda2 uses ext4 (my Linux root filesystem). /dev/sda3 is formatted as Linux swap, which isn't a traditional file system but a dedicated space for virtual memory paging rather than file storage.



Partition	File System	Common OS / Usage	Forensic Importance
/dev/sda1	FAT32 (vfat)	EFI System Partition required by UEFI boot standard on my system	Simple structure, minimal metadata; commonly used on boot/removable media
/dev/sda2	ext4	My Linux root filesystem (Kali)	Inode-based metadata with journaling supports timeline analysis and partial recovery of deleted files
/dev/sda3	Linux swap	Virtual memory paging space	Not a file system itself, but may contain fragments of data swapped from RAM

Why file system identification matters in forensics

Identifying the file system on a volume is one of the first steps an examiner must take, because different file systems store metadata — timestamps, permissions, deletion records, file ownership — in completely different structures and formats. For example, ext4 uses inode-based metadata and a journal that can help reconstruct recent activity or partially recover deleted files, while FAT32 has a much simpler structure with fewer metadata fields and no journaling at all, meaning less forensic detail is recoverable from it. Using the wrong assumptions about a file system — for instance, applying NTFS-specific analysis techniques to an ext4 partition — could cause an examiner to misinterpret timestamps, miss recoverable data, or fail to locate artifacts that exist in a different structure than expected. On my own system, correctly identifying `/dev/sda2` as ext4 rather than assuming a generic or Windows-style file system is what determines which forensic tools and interpretation methods would actually apply if this were a real investigation.

Part G - Final Practical Assignment

Task	Required Evidence	Student Notes / Answer
Identify partition 1	Screenshot of partition table and command output	<code>sudo fdisk -l</code> (Part A2 screenshot) Device <code>/dev/sda1</code> , Type EFI System, Disklabel type gpt
Record partition 1 sector count	Start sector, end sector or sector count	Start: 2048, End: 2000895, Sector count: 1,998,848
Calculate partition 1 size	Formula and result in GB	$1,998,848 \text{ sectors} \times 512 \text{ bytes} = 1,023,410,176 \text{ bytes} \rightarrow \div 1024 = 999,424 \text{ KB} \rightarrow \div 1024 = 976 \text{ MB} \rightarrow \div 1024 = 0.95 \text{ GB}$
Identify partition 2	Screenshot of partition table and command output	<code>sudo fdisk -l</code> (Part A2 screenshot) Device <code>/dev/sda2</code> , Type Linux filesystem, Disklabel type gpt
Record partition 2 sector count	Start sector, end sector or sector count	Start: 2000896, End: 960196607, Sector count: 958,195,712
Calculate partition 2 size	Formula and result in GB	$958,195,712 \text{ sectors} \times 512 \text{ bytes} = 490,596,204,544 \text{ bytes} \rightarrow \div 1024 = 479,097,856 \text{ KB} \rightarrow \div 1024 = 467,869 \text{ MB} \rightarrow \div 1024 = 456.9 \text{ GB}$

Task	Required Evidence	Student Notes / Answer
Compare with OS-reported size	Disk Management, Get-Volume, lsblk, or df output	df -Th (Part F screenshot): /dev/sda1 = 975M, /dev/sda2 = 449G. lsblk (Part A2): sda1 = 976M, sda2 = 456.9G
Explain difference if any	Short explanation of rounding, reserved space, file system overhead, or GiB/GB differences	See below

Explaining the difference:

My manually calculated sizes (0.95 GB for partition 1, 456.9 GB for partition 2) closely match the OS-reported sizes from lsblk (976M and 456.9G) and df -Th (975M and 449G), with only small variations. There are two main reasons for this: first, GiB vs GB units fdisk and my manual calculation use binary units (1 GiB = 1024^3 bytes), while some tools like parted (seen earlier in Part D) reported partition 1 as ~1023MB using decimal units (1 GB = 1,000,000,000 bytes), which explains why the same physical partition shows slightly different numbers depending on which convention a tool uses. Second, file system overhead and reserved space df -Th reports the usable space after the file system's own metadata structures (journal, reserved blocks, superblock, inode tables) are subtracted, which is why /dev/sda2 shows 449G in df but 456.9G in the raw partition table the ~8G difference is space the ext4 file system itself reserves for its own bookkeeping and is not available for user files. This distinction matters forensically: an examiner working from raw sector/partition data (via fdisk) is seeing the true physical partition boundaries, while an examiner working from OS-level tools (df, lsblk mount view) is seeing the usable capacity after formatting overhead both are correct, they're just answering slightly different questions about the same partition.

Conclusion

This lab gave me a much deeper, hands-on understanding of how a computer's physical hardware, firmware, and file systems work together to store and expose forensic evidence. Capturing my own system's specifications in Part A showed me that even basic identification commands `hostname`, `CPU` model, memory size, boot mode form the baseline profile an examiner needs before any deeper analysis begins. Investigating running processes in Part B reinforced that live system state (processes, PIDs, memory) is fundamentally different from data at rest on disk, and that this volatile evidence disappears the moment a system powers off, which is why live triage matters in real investigations.

The sector and partition size calculations in Part C were the most valuable part of this lab for me, because working through the raw math $\text{sectors} \times \text{bytes per sector}$, then converting through KB, MB, and GB showed me exactly what forensic tools are doing internally rather than just trusting their rounded output. Applying that formula to my own real disk, instead of only the textbook example, let me verify that my manual calculations matched what `fdisk`, `parted`, and `lsblk` independently reported, which gave me confidence in both the math and the tools.

Inspecting my partition table safely in Part D, without ever running a write command, taught me the importance of read-only evidence handling a core forensic principle, since altering a disk during examination can destroy or contaminate evidence. Cross-referencing `fdisk`, `parted`, and `blkid` and seeing them all agree on the same partition layout, UUIDs, and file systems reinforced why examiners triangulate findings across multiple independent tools rather than trusting a single source. Understanding the boot process in Part E, and confirming my own system uses UEFI with a GPT partition table rather than Legacy BIOS/MBR, connected the abstract theory directly to evidence I had already gathered showing how each layer, from firmware settings to the boot loader to kernel handoff, could leave its own trace or point of failure that matters forensically.

Finally, identifying my file systems in Part F and verifying partition sizes against real partition-table evidence in Part G tied the whole lab together: I learned that a "size" reported by an OS tool and a size calculated from raw sectors can differ slightly, not because either is wrong, but because of unit conventions (GiB vs GB) and file system overhead a subtlety an examiner must understand to avoid misreporting evidence. Overall, this lab reinforced that digital forensics depends on verifying evidence at the lowest reliable level raw sectors and partition tables rather than assuming a single tool's summary is the full picture.